



國立陽明交通大學資訊工程學系資訊中心

IT Center of Department of Computer Science, National Yang Ming Chiao Tung University

Kubernetes Resources & Networking

hytsao (2026, CC)

Outline

- Kubernetes Object Model & kubectl
- Workload Resources
 - Pod, ReplicaSet, Deployment, DaemonSet, Job, CronJob
- Configuration & Storage
 - ConfigMap, Secret, Volume, PersistentVolume / PersistentVolumeClaim / StorageClass, StatefulSet
- Networking
 - Network Model, Service, DNS / CoreDNS, Ingress
- Health Checks
- Resource Management
 - Requests & Limits, QoS, LimitRange / ResourceQuota

Kubernetes Object Model

Kubernetes Object Model

- Everything in Kubernetes is represented as an Object stored in etcd
- Most Kubernetes manifests you write primarily include...

apiVersion

K8s provides an API that supports this object

kind

Object we are trying to create

metadata

The information about the object. e.g., name, label

spec

Defines what's inside the object we are creating

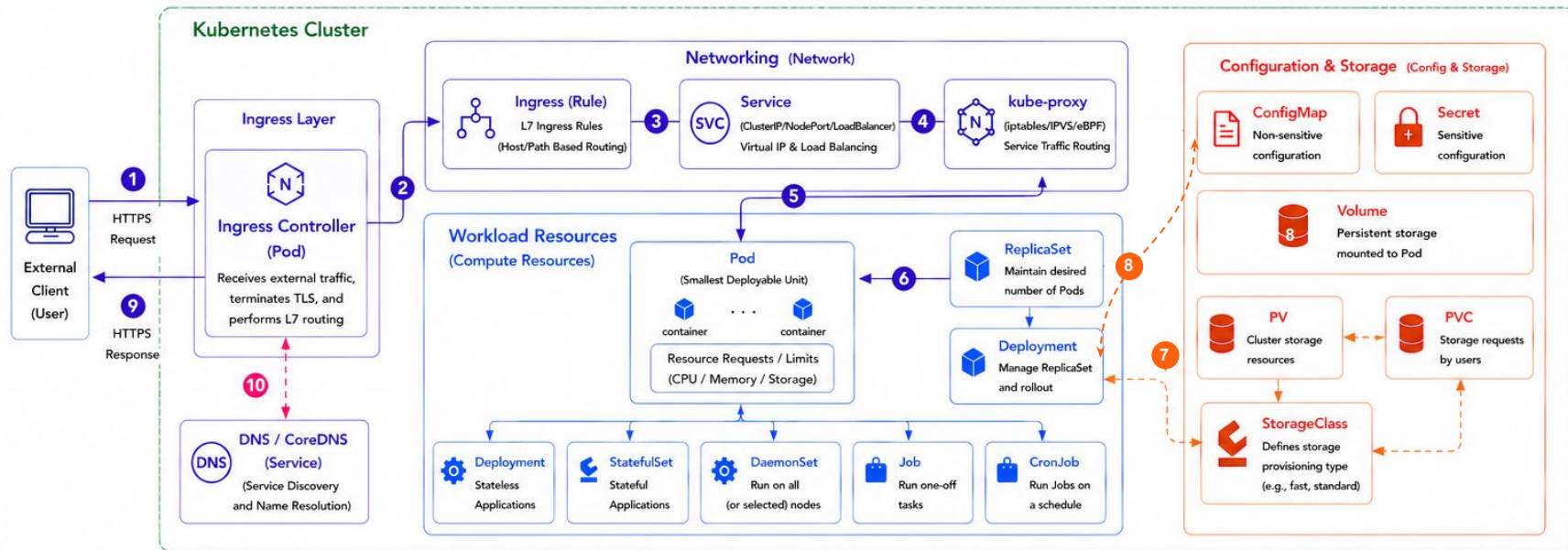
Pod

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: nginx-demo
5   labels:
6     app: web
7 spec:
8   containers:
9     - name: nginx
10       image: "nginx:1.14.2"
11       ports:
12         - containerPort: 80
```

kubectl commands

- **Syntax:** `kubectl [command] [TYPE] [NAME] [flags]`
 - Apply / Create / Delete
 - `kubectl apply -f manifest.yaml` # create or update (idempotent)
 - `kubectl delete -f manifest.yaml` # remove
 - `kubectl delete pod my-pod` # remove by name
 - Inspect
 - `kubectl get pods -n <ns> -o wide` # list with node & IP
 - `kubectl describe pod <name>` # events, conditions, probes
 - `kubectl get events --sort-by=.lastTimestamp`

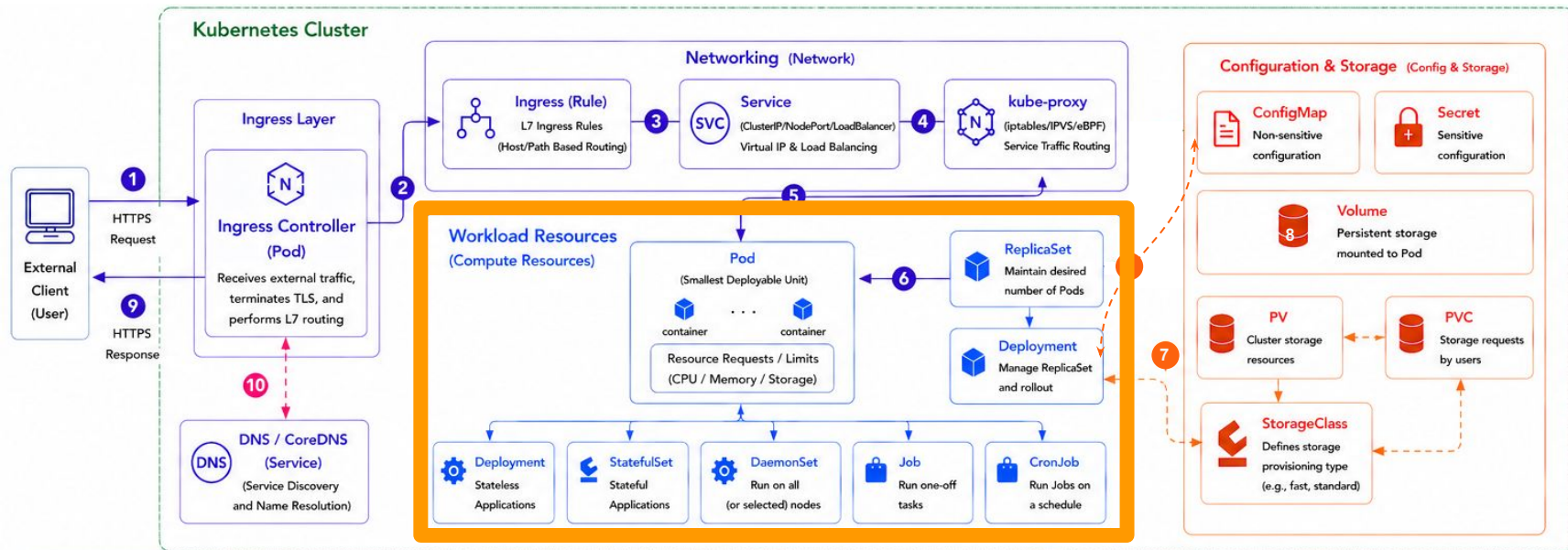
Overview



Workload Resources

Including: Pod, ReplicaSet, Deployment, DaemonSet, Job, CronJob

Overview



Pod

- Pod is the smallest deployable unit in Kubernetes
 - A Pod wraps one or more containers that share resources
- Shared resources inside a Pod
 - Network namespace: same IP address and port space
 - Containers communicate via localhost
 - Storage volumes: containers can read/write the same Volumes
 - Lifecycle: created, restarted, and terminated together
- Key properties
 - Each Pod gets a unique cluster-internal IP (flat network)
 - Pod IPs are ephemeral – they may change when the Pod is recreated
 - Pod spec is mostly immutable once created (delete → recreate to change)

Pod - multiple containers

- Multi-container patterns
 - Sidecar - augments main container (log forwarder, proxy, sync)
 - Init container - runs before main containers, prepares environment
 - Ephemeral - injected temporarily for debugging (`kubectl debug`)
- Pods that run multiple containers that need to work together
- Best practice is to run only one container per pod

Pod - phases

- Pod phases (`kubectl get pod` → `STATUS` column)
 - Pending - accepted by cluster; waiting for scheduling or image pulling
 - Running - at least one container is running
 - Succeeded - all containers exited 0 (Jobs use this)
 - Failed - all containers terminated; at least one non-zero exit
 - Unknown - node is unreachable
- Container states (inside a running Pod)
 - Waiting – not running yet (image pull, init containers pending)
 - Running – executing normally
 - Terminated – exited; check `exitCode` and `reason` field

Pod - restartPolicy and conditions

- restartPolicy – controls container restart behavior
 - Always – always restart on exit (default for long-running apps)
 - OnFailure – restart only on non-zero exit
 - Never – never restart
- Pod conditions (inspect with `kubectl describe`)
 - PodScheduled
 - ContainersReady
 - Initialized
 - Ready
 - Pod healthy (all Readiness probes passing)
 - Only Ready Pods receive traffic

Pod - YAML Manifest

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: hello
5   namespace: testing
6   labels:
7     app: hello
8 spec:
9   containers:
10    - name: hello
11      image: 'busybox:1.28'
12      command:
13        - sh
14        - '-c'
15        - 'echo "Hello, Kubernetes!" && sleep 3600'
16      ports:
17        - containerPort: 80
18      restartPolicy: OnFailure
```

```
# kubectl describe pod hello -n testing

Name:          hello
Namespace:     testing
Node:          na-k8s-w2/172.16.1.6
Labels:        app=hello
Annotations:   cni.projectcalico.org/podIP: 10.244.24.207/32
Status:        Running
IP:            10.244.24.207
Containers:
  hello:
    Container ID:  cri-o://c8a52226220e712d8a929566f60b8529a50a79c488c01a6499f1d54f3c3827a6
    Image:         busybox:1.28
    Image ID:      docker.io/library/busybox@sha256:141c253bc4c3fd0a2
    Port:          80/TCP
    Command:
      sh
      -c
      echo "Hello, Kubernetes!" && sleep 3600
    State:         Running
    Ready:         True
Conditions:
  Type            Status
  Initialized      True
  Ready           True
  ContainersReady True
  PodScheduled     True
Events:
  Type    Reason      Age   From          Message
  ----    -
  Normal  Scheduled   21s   default-scheduler  Successfully assigned testing/hello to na-k8s-w2
  Normal  Pulling     21s   kubelet        spec.containers{hello}: Pulling image "busybox:1.28"
  Normal  Pulled      15s   kubelet        spec.containers{hello}: Successfully pulled image
  Normal  Created     15s   kubelet        spec.containers{hello}: Created container: hello
  Normal  Started     15s   kubelet        spec.containers{hello}: Started container hello
```

Labels, Annotations & Selectors

- **Labels:** key/value pairs attached to objects
 - Used by selectors to group and identify objects.
 - For example: `app=nginx`
- **Annotations:** arbitrary metadata NOT used for selection
 - Larger values allowed (URLs, JSON, etc.)
 - Used by tools: monitoring hints, CI/CD commit SHA, timestamp, etc.
- **Selectors:** query mechanism based on labels
 - Equality. For example: `app=nginx / app!=redis`
 - Set-based. For example: `app in (nginx, redis) / env notin (dev, test)`

- If something needs to be queried → use a **Label**
- If it is informational only → use an **Annotation**

ReplicaSet (rs)

- Ensures a specified number of identical Pod replicas are always running
 - Creates Pods when count falls below desired; removes extras
 - Uses a label selector to identify "owned" Pods
- How selectors work
 - `spec.selector.matchLabels` MUST match `spec.template.metadata.labels`
 - K8s can also adopt existing Pods that match the selector
- Limitation: no built-in rolling update
- Use Deployment instead – it manages ReplicaSets for you
 - Deployment adds rolling update, rollback, and revision history

ReplicaSet - YAML Manifest

- replicas: desired count
- selector must exactly match template labels
- template is a Pod spec without apiVersion and kind
- Scaling:
 - `kubectl scale rs/nginx-rs --replicas=5`

```
1 apiVersion: apps/v1
2 kind: ReplicaSet
3 metadata:
4   name: nginx-rs
5 spec:
6   replicas: 3
7   selector:
8     matchLabels:
9       app: nginx
10  template:
11    metadata:
12      labels:
13        app: nginx
14    spec:
15      containers:
16        - name: nginx
17          image: 'nginx:1.25'
18          ports:
19            - containerPort: 80
20      resources:
21        requests:
22          cpu: 100m
23          memory: 64Mi
24        limits:
25          cpu: 500m
26          memory: 128Mi
```


Deployment (deploy)

- Manages ReplicaSets and adds declarative rolling update / rollback
 - When you update the Pod spec, Deployment creates a new ReplicaSet
 - Gradually scales up new ReplicaSet while scaling down old ReplicaSet
- Update strategies
 - RollingUpdate (default) – zero-downtime upgrade
 - maxUnavailable: max Pods that can be down during update (default 25%)
 - maxSurge: max extra Pods above desired count (default 25%)
 - Recreate – kill **ALL** old Pods first, then create new ones (causes downtime)
- minReadySeconds: seconds new Pod must be Ready before counted available

Deployment (deploy)

- Rollback
 - K8s keeps revision history (revisionHistoryLimit, default 10)
 - `kubectl rollout history deployment/<name>`
 - `kubectl rollout undo deployment/<name>` → previous revision
 - `kubectl rollout undo deployment/<name> --to-revision=2`
- Best suited for **stateless** applications
 - Pods are interchangeable; no stable identity required
 - Typical: web servers, REST APIs, background workers

Deployment - YAML Manifest

- replicas: 3
- revisionHistoryLimit: 5
- minReadySeconds: 10
- selector must exactly match template labels
- strategy.rollingUpdate
 - maxUnavailable: 1
 - maxSurge: 1
 - minReadySeconds: 10
 - wait 10s after Ready before proceeding

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: nginx-deploy
5   labels:
6     app: nginx
7 spec:
8   replicas: 3
9   revisionHistoryLimit: 5
10  minReadySeconds: 10
11  selector:
12    matchLabels:
13      app: nginx
14  strategy:
15    type: RollingUpdate
16    rollingUpdate:
17      maxUnavailable: 1
18      maxSurge: 1
19  template:
20    metadata:
21      labels:
22        app: nginx
23    spec:
24      containers:
25        - name: nginx
26          image: 'nginx:1.25'
27          ports:
28            - containerPort: 80
29          resources:
30            requests:
31              cpu: 100m
32              memory: 64Mi
33            limits:
34              cpu: 500m
35              memory: 128Mi
```

DaemonSet (ds)

- Ensures exactly ONE Pod runs on every (or selected) Node
 - Node joins cluster → DaemonSet Pod automatically scheduled on it
 - Node removed → its DaemonSet Pod is garbage collected
- Typical use cases
 - Log collection: Fluentd, Filebeat, Promtail
 - Metrics collection: Prometheus Node Exporter, Datadog Agent
 - Network plugin: kube-proxy itself, CNI plugins (Calico, Cilium)
 - Storage driver: Ceph CSI, host volume management

DaemonSet (ds)

- No replicas field – count is determined by node count
- Update strategies
 - RollingUpdate (default) – replaces Pods one by one
 - OnDelete – replace only when Pod is manually deleted
- DaemonSet vs Deployment
 - Deployment: "I want N replicas anywhere"
 - DaemonSet: "I want one on EACH node"

[App.] DaemonSet - YAML Manifest

- No replicas field
- tolerations able to run on
 - control-plane or other tainted nodes

```
1 apiVersion: apps/v1
2 kind: DaemonSet
3 metadata:
4   name: fluentd-elasticsearch
5   namespace: logging
6   labels:
7     k8s-app: fluentd
8 spec:
9   selector:
10    matchLabels:
11      k8s-app: fluentd
12   template:
13     metadata:
14       labels:
15         k8s-app: fluentd
16     spec:
17       containers:
18         - name: fluentd-elasticsearch
19           image: 'quay.io/fluentd_elasticsearch/fluentd:v5.0.1'
20           volumeMounts:
21             - name: varlog
22               mountPath: /var/log
23       volumes:
24         - name: varlog
25           hostPath:
26             path: /var/log
```

[App.] Job - One-Shot Tasks

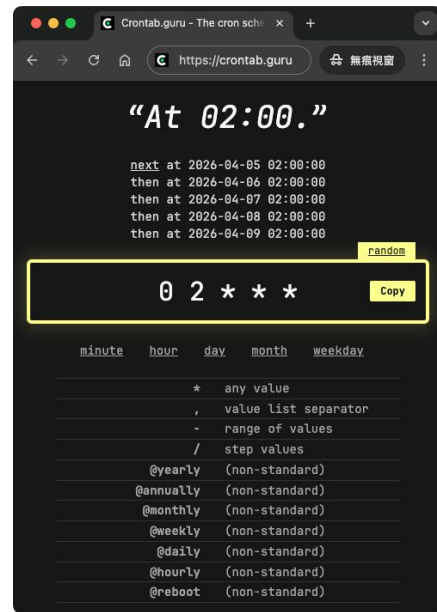
- Runs a Pod to completion – task finishes, Pod is not restarted
 - Unlike Deployment, completion is success; K8s tracks it
- Key spec fields
 - completions – how many successful completions needed (default 1)
 - parallelism – how many Pods can run simultaneously (default 1)
 - backoffLimit – max retries before Job is marked Failed (default 6)
 - activeDeadlineSeconds – hard wall-clock limit
 - ttlSecondsAfterFinished – auto-clean up Job after N seconds
- restartPolicy must be OnFailure or Never (cannot be Always)

[App.] Job - One-Shot Tasks

- Completion modes
 - NonIndexed (default) – any Pod completion counts
 - Indexed – each Pod gets index 0...N-1, useful for work-queue sharding
- Use cases
 - Database schema migration before deploying new app version
 - Batch data processing (parallel workers across large dataset)
 - One-time report generation or file conversion

[App.] CronJob (cron) – Scheduled Tasks

- Creates a Job on a cron schedule
 - Cron syntax: "minute hour day month weekday"
 - "0 2 * * *" = every day at 02:00
 - "* /5 * * * *" = every 5 minutes
- Key spec fields
 - schedule – cron expression (UTC by default)
 - timeZone – e.g. "Asia/Taipei" (K8s 1.27+)
 - concurrencyPolicy – behavior when previous Job is still running
 - Allow (default) / Forbid / Replace
 - startingDeadlineSeconds – skip if Job cannot start within N seconds
 - successfulJobsHistoryLimit (default 3) / failedJobsHistoryLimit (default 1)



<https://crontab.guru/>

[App.] CronJob (cron) – Scheduled Tasks

- Use cases
 - Database backup (nightly)
 - Cache warm-up / stale entry cleanup
 - Scheduled reports, notification emails
- Debug tip: trigger manually without waiting for the schedule
 - `kubectl create job test --from=cronjob/<name>`

[App.] Job & CronJob - YAML Manifest

- Job
 - restartPolicy: OnFailure
 - backoffLimit: 3
 - ttlSecondsAfterFinished auto cleanup
- CronJob
 - wraps a jobTemplate which is a Job spec
 - concurrencyPolicy:
 - Forbid prevents overlapping runs

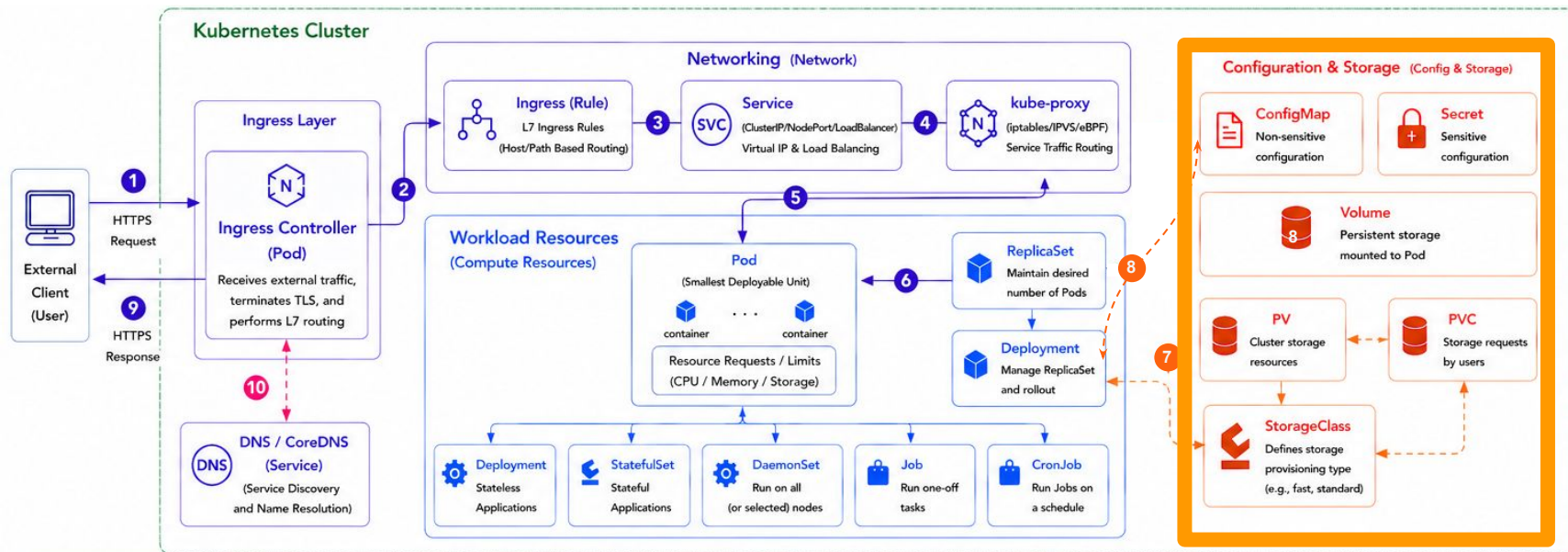
```
1 apiVersion: batch/v1
2 kind: Job
3 metadata:
4   name: db-migration
5 spec:
6   completions: 1
7   parallelism: 1
8   backoffLimit: 3
9   ttlSecondsAfterFinished: 600
10  template:
11    spec:
12      restartPolicy: OnFailure
13      containers:
14      - name: migrate
15        image: 'myapp:latest'
16        command:
17          - python
18          - manage.py
19          - migrate
```

```
1 apiVersion: batch/v1
2 kind: CronJob
3 metadata:
4   name: db-migration
5 spec:
6   schedule: 0 2 * * *
7   timeZone: Asia/Taipei
8   concurrencyPolicy: Forbid
9   successfulJobsHistoryLimit: 3
10  failedJobsHistoryLimit: 1
11  jobTemplate:
12    spec:
13      completions: 1
14      parallelism: 1
15      backoffLimit: 3
16      ttlSecondsAfterFinished: 600
17      template:
18        spec:
19          restartPolicy: OnFailure
20          containers:
21          - name: migrate
22            image: 'myapp:latest'
23            command:
24              - python
25              - manage.py
26              - migrate
```

Configuration & Storage

Including: ConfigMap, Secret, Volume, PV / PVC / StorageClass

Overview



ConfigMap (cm) – Non-Sensitive data

- Stores non-sensitive config as key-value pairs
 - Decouples configuration from the container image
 - Can store simple values, multi-line text, or entire config files
- Size limit: 1 MiB per ConfigMap
- `kubectl create configmap my-cm --from-file=config.yaml`
- ConfigMap YAML Manifest
 - data:
 - key-value
 - | for multi-line

```
1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: app-config
5 data:
6   APP_ENV: production
7   LOG_LEVEL: info
8   nginx.conf: |
9     server {
10       listen 80;
11       location / { proxy_pass http://backend; }
12     }
```

Secret – Sensitive Data

- Same methods as ConfigMap, but intended for passwords, tokens, TLS certificates, SSH keys
- Secret data is **NOT encrypted** by default
 - Values are base64-encoded (encoding ≠ encryption)
 - Anyone who can GET the Secret object sees the decoded data
 - Stored as base64 in etcd (plaintext unless etcd encryption enabled)
- Production hardening
 - Enable etcd encryption-at-rest
 - Use RBAC to restrict who can read Secrets (least privilege)
 - Consider external secret stores
 - HashiCorp Vault, AWS Secrets Manager, Azure Key Vault
 - External Secrets Operator, Sealed Secrets

Secret – secret types

- Common Secret types
 - Opaque – generic, user-defined key-value pairs
 - kubernetes.io/tls – TLS certificate (tls.crt) + key (tls.key)
 - kubernetes.io/dockerconfigjson – image pull credentials
- Secret YAML Manifest
 - stringData:
 - auto base64-encodes
 - data:
 - needs pre-encoded values

```
1 # format 1
2 apiVersion: v1
3 kind: Secret
4 metadata:
5   name: db-secret
6 type: Opaque
7 stringData:
8   DB_USER: admin
9   DB_PASSWORD: s3cr3t!
```

```
1 # format 2
2 apiVersion: v1
3 kind: Secret
4 metadata:
5   name: db-secret
6 type: Opaque
7 data:
8   DB_USER: YWRtaW4=
9   DB_PASSWORD: czNjcjN0IQ==
```


Volumes

- The mechanism Kubernetes uses to solve data storage and data sharing
- Why volumes
 - Data persistence: Container filesystem is ephemeral (wiped on container restart)
 - Shared storage: Multiple containers in a Pod need to share data
- Volumes type, according to "Life expectancy"
 - Ephemeral Volumes
 - Data deleted when Pod is deleted
 - Use case: caching services
 - Persistent Volumes
 - Data persists even if the Pod is deleted
 - Use case: database, logs

Volumes

- Volumes type, according to "Implementation approach"
 - emptyDir – Temporary storage created on Pod startup
 - Tied to Pod lifetime
 - Data persists across container restarts but deleted on Pod removal
 - Storage: Local disk (default) or RAM (medium: Memory)
 - Use Cases: Scratch space (merge sort), shared buffer for sidecars, caching
 - hostPath – Mounts a file/directory from the Host Node's filesystem
 - Breaks Portability (Node-dependent) and poses Security risks
 - Use Cases: DaemonSets needing system logs (/var/log) or node internals

Volumes

- Volumes type, according to "Implementation approach"
 - configMap / secret – Injects cluster data as read-only files
 - "Projects" keys from etcd into the container filesystem
 - Hot-sync: Files auto-update (~1 min) when source changes
 - Except for **subPath** mounts
 - Security: Secrets use tmpfs (RAM-backed) to avoid writing to Node disk
 - downwardAPI – Makes Pod/Container metadata available to processes
 - Exposes Pod and Container metadata as files
 - e.g., metadata.name, status.podIP, requests.cpu
 - Benefit: Enables "environment-aware" apps without an API client

Volumes

- Volumes type, according to "Implementation approach"
 - NFS – Network File System mount
 - Support: ReadWriteMany (multiple Nodes can access simultaneously)
 - Persistence: Data remains even after the Pod is deleted
 - Use case: legacy app data sharing, centralized file storage
 - CSI (Container Storage Interface) – Modern standard for external storage
 - Nature: Replaces legacy "In-tree" drivers
 - Capabilities: Volume snapshots, resizing, and vendor-specific storage (AWS, GCP, Azure)

[App.] Configure a Pod to Use a Configuration files

- Two ways to consume a Configuration in a Pod
 - As environment variables
 - Inject with
 - `envFrom.[configMapRef|secretRef]` - inject ALL keys as env vars
 - `env[ ].valueFrom.[configMapKeyRef|secretKeyRef]` - inject ONE specific key (recommended)
 - Command-line arguments via `$(ENV_VAR)` substitution
 - Volume mount with files
 - Each key becomes a file in the mounted directory
 - Best for config files: `nginx.conf`

[App.] Pod YAML Manifest with Ephemeral Volumes

- As environment variables
- Volume mount with files
 - mount one key as a specific file

```
1 # format 1
2 apiVersion: v1
3 kind: Secret
4 metadata:
5   name: db-secret
6 type: Opaque
7 stringData:
8   DB_USER: admin
9   DB_PASSWORD: s3cr3t!
```

```
1 apiVersion: v1
2 kind: ConfigMap
3 metadata:
4   name: app-config
5 data:
6   APP_ENV: production
7   LOG_LEVEL: info
8   nginx.conf: |
9     server {
10       listen 80;
11       location / { proxy_pass http://backend; }
12     }
```

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: cm-demo-pod
5 spec:
6   containers:
7   - name: app
8     image: 'myapp:latest'
9     command:
10     - /bin/sh
11     - '-c'
12     args:
13     - >-
14       echo 'Starting in ${APP_ENV} mode with log level ${MY_LOG_LEVEL}';
15       /entrypoint.sh
16   env:
17   - name: MY_LOG_LEVEL
18     valueFrom:
19       configMapKeyRef:
20         name: app-config
21         key: LOG_LEVEL
22   envFrom:
23   - configMapRef:
24     name: app-config
25   - secretRef:
26     name: db-secret
27   volumeMounts:
28   - name: cfg-vol
29     mountPath: /etc/nginx/nginx.conf
30     subPath: nginx.conf
31   volumes:
32   - name: cfg-vol
33     configMap:
34       name: app-config
```

Persistent volumes

- Persistent volumes
 - Data persists even if the Pod is deleted
 - Decoupling of storage supply and demand
 - PersistentVolume (PV) – Supply a "Resource" in the cluster
 - PersistentVolumeClaim (PVC) – Demand a "Request" for storage

PersistentVolume (PV)

- **PersistentVolume (PV) – Supply a "Resource" in the cluster**
 - A piece of storage in the cluster that has been provisioned
 - Scope: Cluster-wide (not namespaced)
 - Its lifecycle is independent of any individual Pod that uses the PV
 - Captures the details of the implementation, be it NFS, iSCSI, or a cloud-provider-specific storage system (via CSI)
- **Provisioning Types:**
 - Static Provisioning: A cluster administrator creates several PVs manually
 - Dynamic Provisioning: Via StorageClasses (defined below)

[App.] PersistentVolume - YAML Manifest

```
1 apiVersion: v1
2 kind: PersistentVolume
3 metadata:
4   name: pv-demo
5 spec:
6   capacity:
7     storage: 10Gi
8   accessModes:
9     - ReadWriteOnce
10  reclaimPolicy: Retain
11  storageClassName: standard
12  hostPath:
13    path: /mnt/data
```

PersistentVolumeClaim (PVC)

- **PersistentVolumeClaim (PVC) – Demand a "Request" for storage**
 - Pods use PVCs as volumes. The cluster inspects the claim to find the bound volume and mounts that volume for the Pod (Binding)
 - Allows a developer to request specific size and access modes (e.g., ReadWriteOnce, ReadOnlyMany, ReadWriteMany) without knowing the underlying storage provider
 - Scope: Namespaced (exists within a Namespace)

[App.] PersistentVolumeClaim - YAML Manifest

```
1 apiVersion: v1
2 kind: PersistentVolumeClaim
3 metadata:
4   name: pod-demo-pvc
5 spec:
6   storageClassName: standard
7   accessModes:
8     - ReadWriteOnce
9   resources:
10     requests:
11       storage: 10Gi
```

StorageClass (SC)

- **StorageClass – The "Blueprint" for dynamic provisioning**
 - Allows admin to describe the "classes" of storage they offer
 - e.g., "fast-ssd" vs "standard-hdd"
 - Dynamic Provisioning
 - Enables storage volumes to be created on-demand
 - When a PVC requests a specific StorageClass, the cluster automatically provisions the backend storage and creates the PV

[App.] StorageClass - YAML Manifest

```
1 apiVersion: storage.k8s.io/v1
2 kind: StorageClass
3 metadata:
4   name: standard
5 provisioner: kubernetes.io/aws-ebs
6 parameters:
7   type: gp3
8 reclaimPolicy: Delete
9 volumeBindingMode: WaitForFirstConsumer
10 allowVolumeExpansion: true
```

[App.] Persistent volumes supplement

- Access Modes (can specify one or more access modes)
 - ReadWriteOnce (RWO): Mounted as read-write by a single node.
 - ReadOnlyMany (ROX): Mounted read-only by many nodes.
 - ReadWriteMany (RWX): Mounted as read-write by many nodes (common for NFS).
- Reclaim Policy
 - Retain: Keeps the data after the PVC is deleted, allowing for manual recovery
 - Delete: Automatically removes the PV and the associated storage asset from the external infrastructure (e.g., AWS EBS or GCP PD)
 - Recycle: Performs a basic scrub (`rm -rf /thevolume/*`) to make the volume available again (deprecated in favor of dynamic provisioning)

[App.] StatefulSet (sts)

- Deployment Pods are interchangeable – no stable identity
- Stateful apps need guaranteed identity across rescheduling
 - Databases (MySQL, Postgres), message brokers (Kafka), caches (Redis Cluster)
- StatefulSet provides three guarantees
 - Stable Pod name: `app-0`, `app-1`, `app-2` → survive restarts
 - Stable DNS per Pod: `app-0.<headless-svc>.<namespace>.svc.cluster.local`
 - Stable storage: each Pod binds to its own dedicated PersistentVolumeClaim (PVC)
 - PVC survives Pod deletion → data is NOT lost on reschedule
- Ordered operations
 - Scale up: `app-0` must be Ready before `app-1` is created
 - Scale down: `app-2` is deleted first, then `app-1`, then `app-0`
 - Safe for leader election and replication bootstrapping

[App.] StatefulSet - YAML

- serviceName → links to the Headless Service
- Headless Service
 - clusterIP: None
 - enables per-Pod DNS
- volumeClaimTemplates
 - auto-creates PVCs

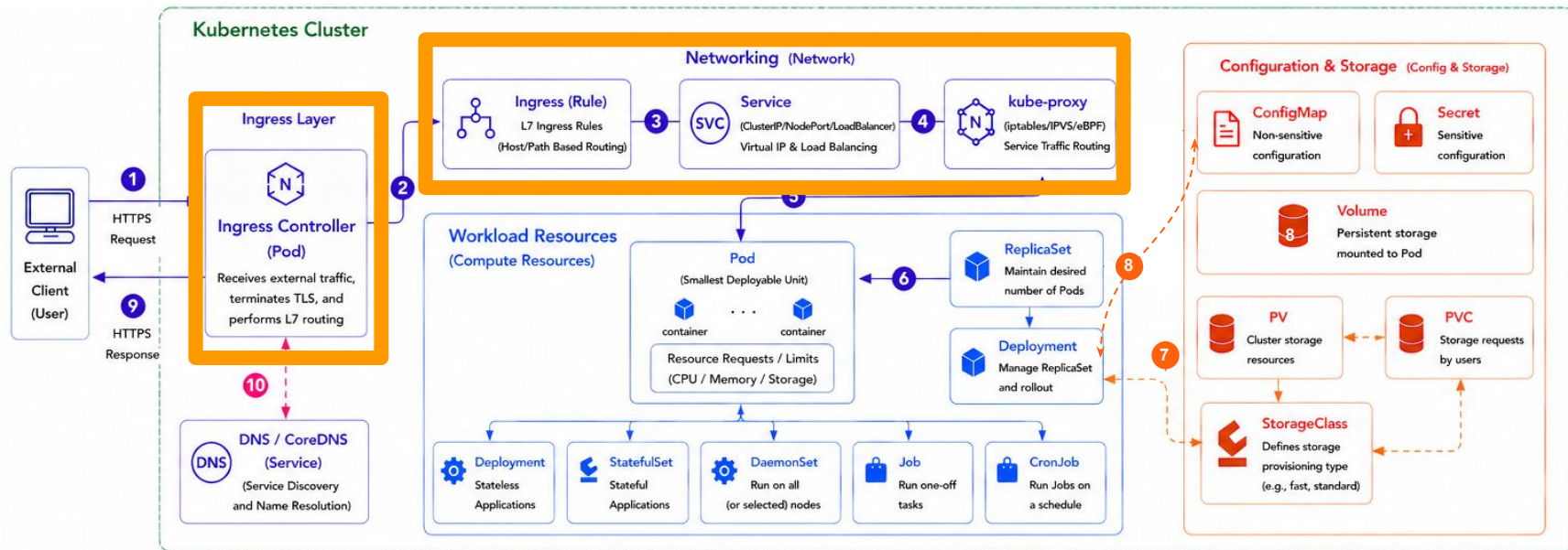
```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: mysql
5 spec:
6   clusterIP: None
7   selector:
8     app: mysql
9   ports:
10    - port: 3306
```

```
1 apiVersion: apps/v1
2 kind: StatefulSet
3 metadata:
4   name: mysql
5 spec:
6   selector:
7     matchLabels:
8       app: mysql
9   serviceName: mysql
10  replicas: 3
11  template:
12    metadata:
13      labels:
14        app: mysql
15    spec:
16      containers:
17        - name: mysql
18          image: 'mysql:8.0'
19          volumeMounts:
20            - name: data
21              mountPath: /var/lib/mysql
22  volumeClaimTemplates:
23    - metadata:
24        name: data
25      spec:
26        accessModes:
27          - ReadWriteOnce
28        storageClassName: my-storage-class
29      resources:
30        requests:
31          storage: 1Gi
```


Networking

Including: Network Model, Service, DNS / CoreDNS, Ingress, NetworkPolicy

Overview



Kubernetes Network Model

- K8s expects a flat, routable Pod network model, which CNIs implement
 - Every Pod gets a unique cluster-wide IP address
 - Any Pod can communicate with any other Pod without NAT
 - Any Node can communicate with any Pod without NAT
- Implemented by CNI (Container Network Interface) plugins
 - Flannel – simple VXLAN overlay; does NOT support NetworkPolicy
 - Calico – BGP or VXLAN; supports NetworkPolicy; high performance
 - Cilium – eBPF-based; advanced observability, L7 policy, no kube-proxy
 - Weave Net – encrypted overlay mesh

Kubernetes Network Model

- Three distinct network planes
 - Node network – physical/VM network Nodes communicate on
 - Pod network – virtual overlay for Pods (e.g. 10.42.0.0/16)
 - Service network – virtual IP range for Services (e.g. 10.43.0.0/16)
- Pod IPs are ephemeral – they may change when the Pod is recreated
 - You should never hardcode Pod IPs
 - This is exactly why the Service abstraction exists

Service (svc) – The Stable Entry Point

- Service provides a stable IP + DNS name for a dynamic group of Pods
 - Pods come and go (ephemeral IPs); Service IP never changes
 - Uses **label selectors** to dynamically identify healthy target Pods
- Internal mechanics – how traffic actually flows
 - Create Service object → K8s assigns a ClusterIP (virtual IP)
 - EndpointSlice controller watches Pods/Services and maintains EndpointSlice objects
 - EndpointSlices represent backend endpoints for a Service, including endpoint addresses and ports
 - kube-proxy on every Node reads Service + EndpointSlices
 - kube-proxy writes nftables / iptables rules on each Node
 - Traffic → ClusterIP → DNAT (iptables) → real Pod IP

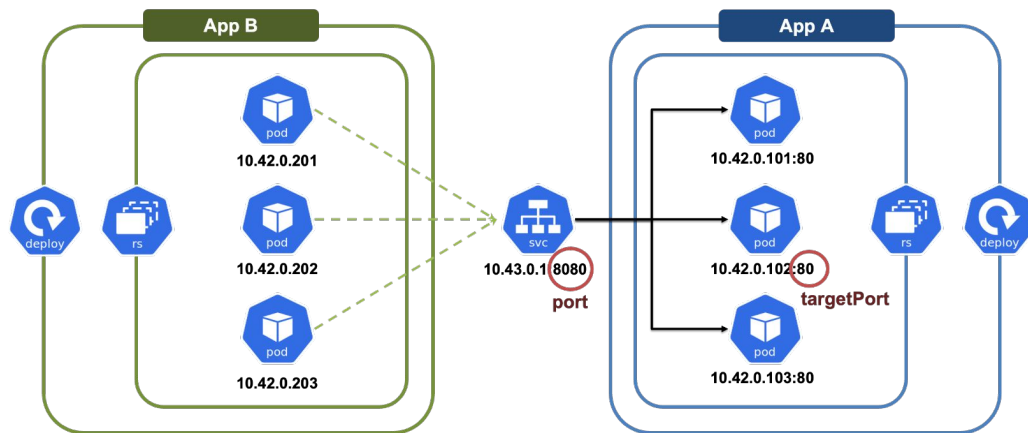
Service (svc)

- ClusterIP is a virtual IP – does not exist on any NIC
 - Lives only in iptables/ipvs rules in the kernel
 - That is why ping ClusterIP usually yields no response
- Service Types
 - ClusterIP (default)
 - NodePort
 - LoadBalancer
 - ExternalName
 - Headless (`clusterIP: None`)

Service (svc) - ClusterIP

- The default type of service
- Allocates a stable virtual IP reachable only inside the cluster
- DNS: `<serviceName>.<namespace>.svc.cluster.local` → ClusterIP
- Use for: inter-service communication, internal APIs

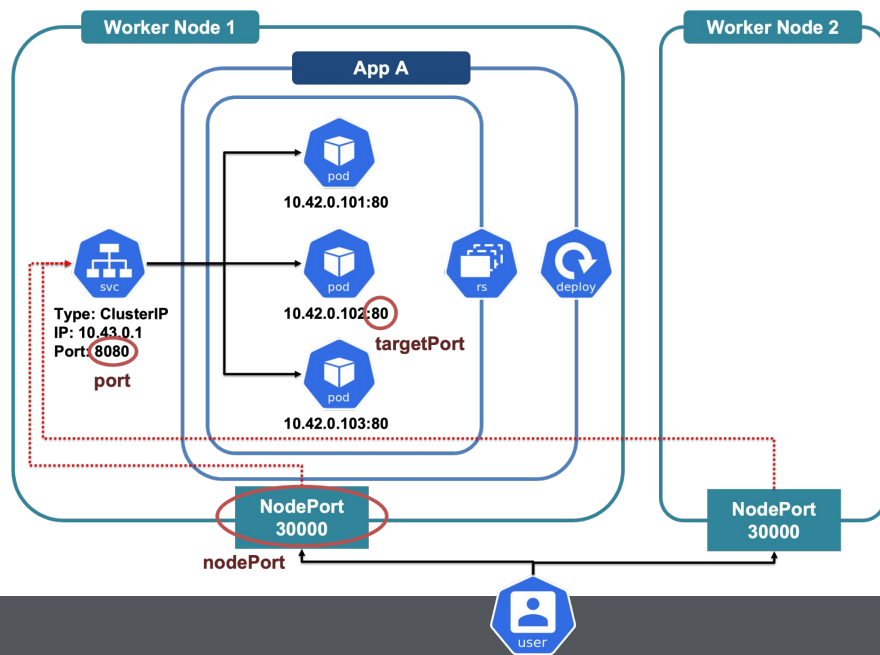
```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: nginx-svc
5 spec:
6   type: ClusterIP
7   selector:
8     app: nginx
9   ports:
10  - port: 8080    # Service port (stable)
11    targetPort: 80 # Pod port
```



Service (svc) - NodePort

- Exposes Service on a fixed port (30000-32767) on EVERY Node
- External access: `<any-NodeIP>:<nodePort>`
- K8s auto-routes to ClusterIP → Pod
- Use for: dev/test, on-prem without cloud LB

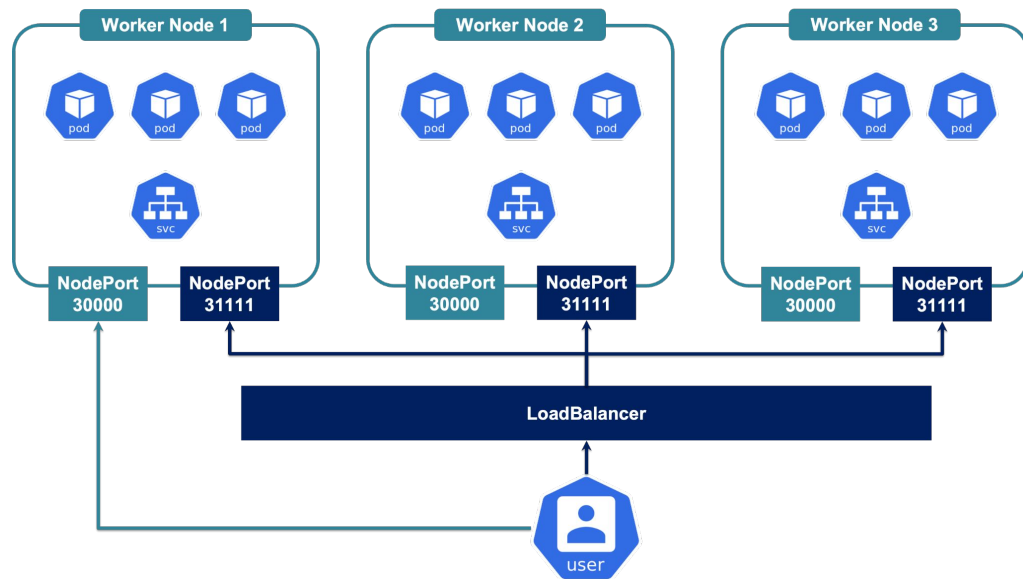
```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: nginx-nodeport
5 spec:
6   type: NodePort
7   selector:
8     app: nginx
9   ports:
10    - port: 8080      # Service port (stable)
11      targetPort: 80  # Pod port
12      nodePort: 30080 # Node port fixed or omit
```



Service (svc) - LoadBalancer

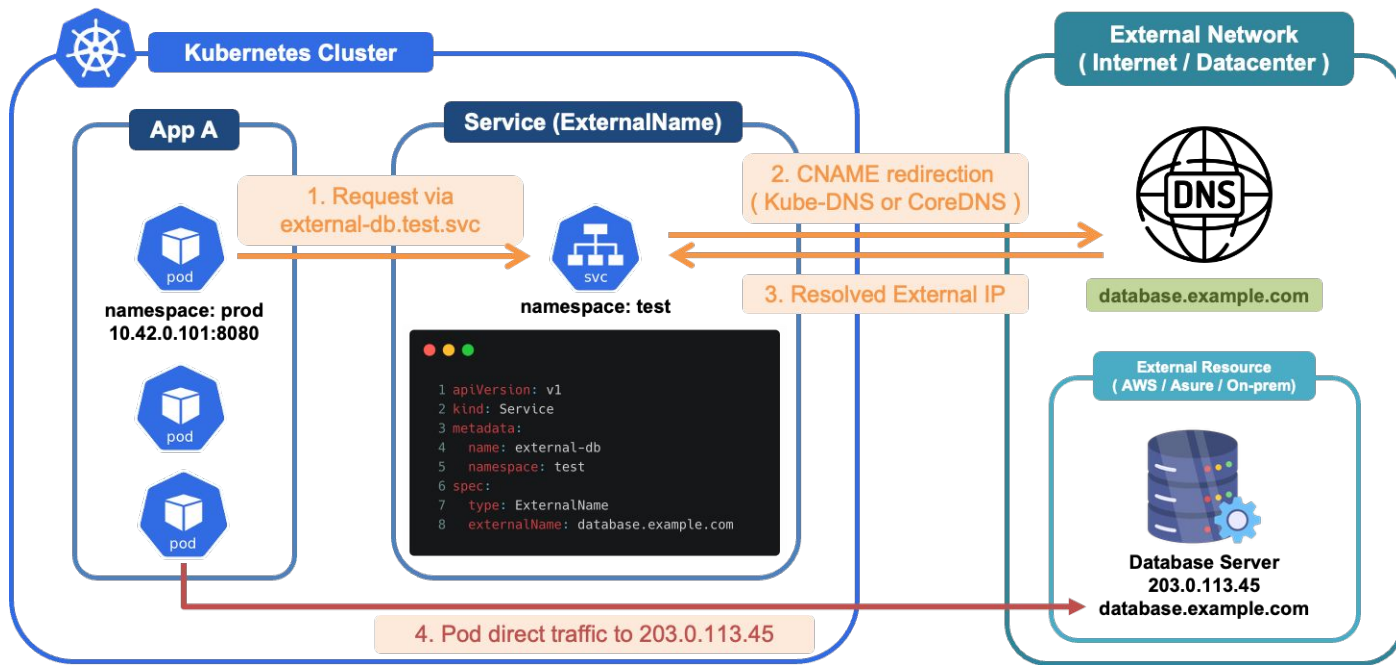
- Extends NodePort; additionally provisions a cloud load balancer
- Cloud controller creates a public IP / DNS and routes traffic to NodePort
- On-premise: use MetalLB to emulate cloud LB behavior

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: nginx-loadbalancer
5 spec:
6   type: LoadBalancer
7   selector:
8     app: nginx
9   ports:
10  - port: 8080      # Service port (stable)
11    targetPort: 80  # Pod port
```



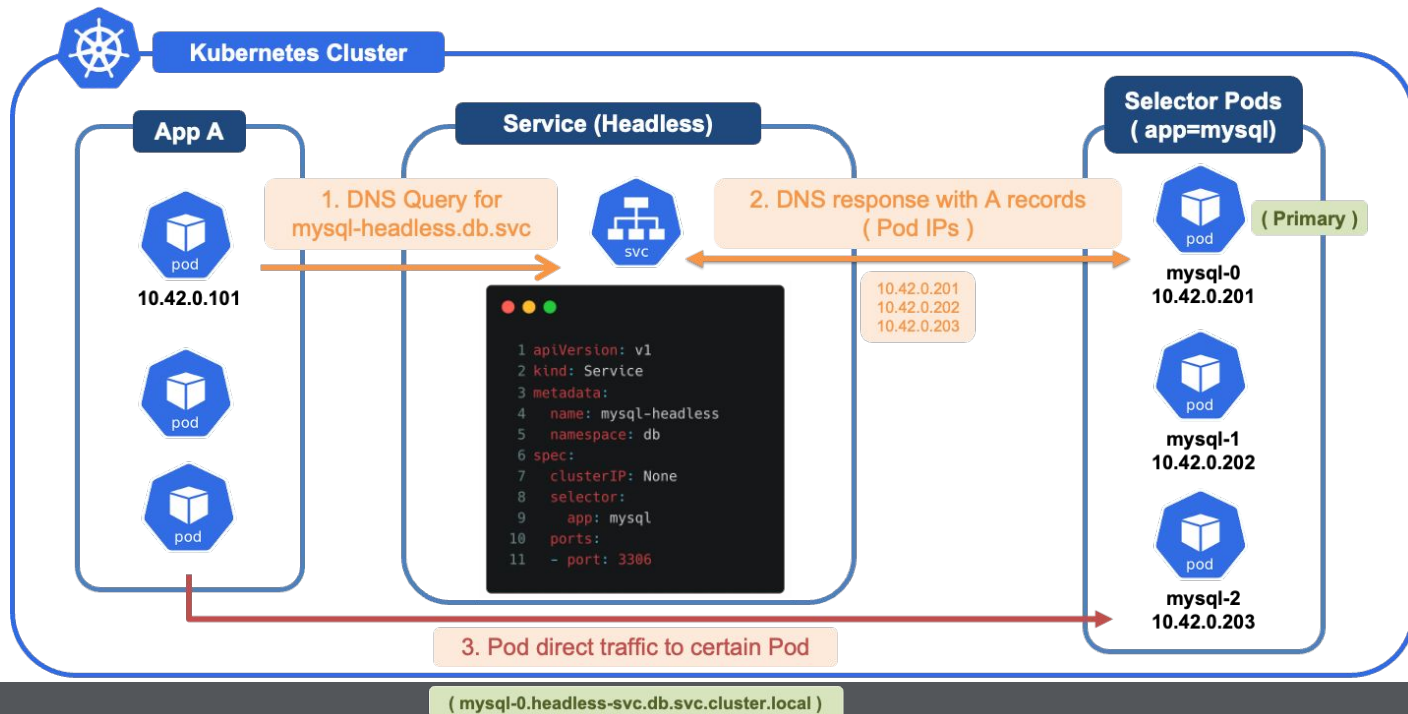
[App.] Service (svc) - ExternalName

- CNAME alias to an external DNS name (no proxying, no ClusterIP)
- Use: access external DB as a K8s Service name (migration-friendly)



[App.] Service (svc) - Headless

- No VIP; DNS returns individual Pod IPs
- Required by StatefulSet for per-Pod stable DNS

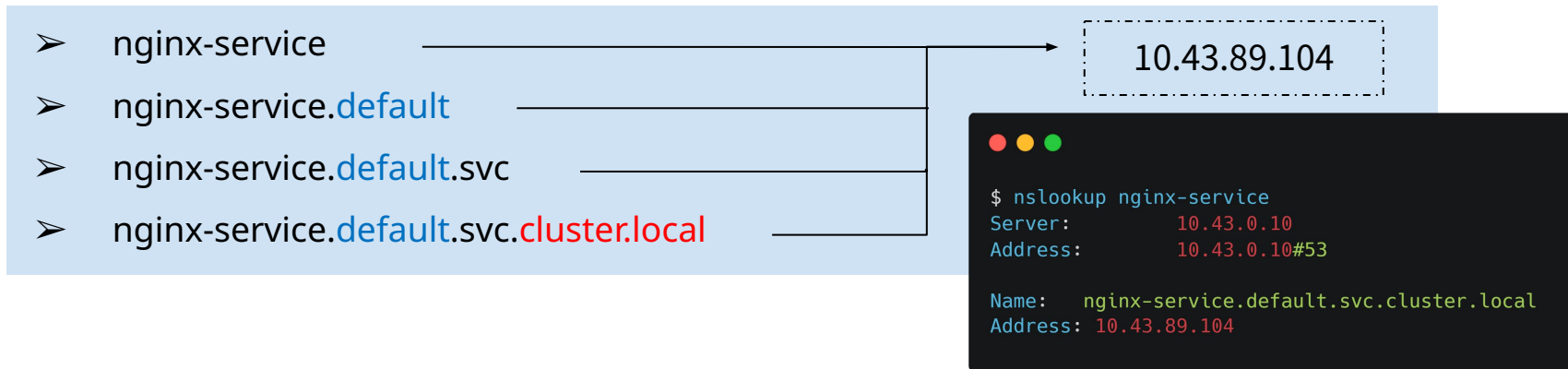


Kubernetes DNS & CoreDNS

- CoreDNS is the default cluster DNS server
 - Runs as a Deployment in kube-system namespace
 - Every Pod's /etc/resolv.conf points to CoreDNS ClusterIP
- DNS record formats
 - Service: `<serviceName>.<namespace>.svc.cluster.local` → ClusterIP (A record)
- CoreDNS config is in ConfigMap coredns in kube-system
 - `kubectl -n kube-system edit cm coredns`

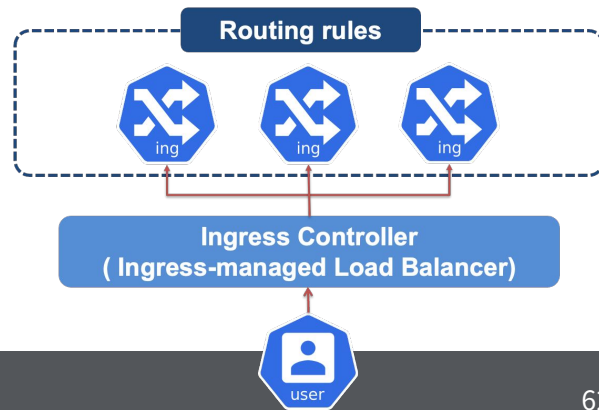
[App.] Kubernetes DNS & CoreDNS

- Short-name resolution (search domain chaining)
 - Pod in "prod" namespace can reach "api-service" without FQDN
 - resolv.conf search list adds:
 - `prod.svc.cluster.local → svc.cluster.local → cluster.local`



Ingress

- Ingress vs Service
 - Service: operate at Layer 4, one public IP per Service (expensive)
 - Ingress: operates at Layer 7, ONE entry point (Ingress controller) routes to many Services
- Ingress refers to routing rules, manages external HTTP/HTTPS access to internal Services
 - Host-based routing: `api.example.com` → `api-service`
 - Path-based routing: `/api` → `api-service`, `/web` → `web-service`
- TLS termination: handles HTTPS, forwards plain HTTP internally
- Ingress requires an Ingress Controller to function



Ingress

- Ingress Controller – the actual implementation
 - A set of pods that watch Ingress resources and implement the routing rules
 - It acts as a reverse proxy and load balancer for HTTP/HTTPS traffic into the cluster
 - Popular Ingress Controllers include: NGINX, Traefik, HAProxy, Istio Ingress Gateway, etc.
 - Allow Multiple controllers co-exist in a cluster
 - `ingressClassName` field selects which controller handles this Ingress



[App.] Ingress - YAML Manifest

- ingressClassName
 - which controller handles this Ingress
- rules → host → paths
 - pathType: Prefix
 - backend: api-service

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: ingress-demo
5 spec:
6   ingressClassName: nginx
7   rules:
8   - host: "api.example.com"
9     http:
10      paths:
11      - pathType: Prefix
12        path: "/api"
13        backend:
14          service:
15            name: api-service
16            port:
17              number: 80
```


Health Checks - Probes

Health Checks

- Kubernetes Probe
 - Health Check Mechanism: A built-in diagnostic tool to monitor container status
 - Automated Monitoring: Periodically audits containers to ensure they are functioning as expected.
- kubelet performs probes to monitor container health
- Three types of probes:
 - Liveness Probe
 - Readiness Probe
 - Startup Probe

Health Checks - Three Types of Probes

- **Liveness Probe** – "Is the container still alive?"
 - Failure action: kill and restart the container
 - Use for: deadlocks, infinite loops, hung processes
 - Warning: do NOT include external dependencies
 - If your DB is down, you don't want your backend Pod restarted

Health Checks - Three Types of Probes

- **Readiness Probe** – "Is the container ready to serve traffic?"
 - Failure action: remove Pod from Service backends (no restart)
 - Traffic stops until probe passes again
 - Use for: startup warm-up, temporary overload, DB initialization
 - Critical for zero-downtime rolling updates
 - New Pod added to Service backends only after Readiness passes
- **Startup Probe** – "Has the app finished starting up?"
 - Disables Liveness + Readiness until it passes once
 - Prevents premature restarts for slow-starting apps (JVM, etc.)
 - $\text{periodSeconds} \times \text{failureThreshold} = \text{max startup time allowed}$
- Minimum recommendation: **always** define a Readiness probe

Health Checks - Probe Mechanisms

- Four probe mechanisms
 - **httpGet**
 - HTTP request; 2xx/3xx response = success
 - Fields: path, port, httpHeaders, scheme (HTTP/HTTPS)
 - **exec**
 - run command inside container; exit code 0 = success
 - Use when there is no HTTP endpoint
 - **tcpSocket**
 - TCP connection established = success
 - Use for non-HTTP services: databases, gRPC, AMQP
 - **grpc**
 - gRPC Health Checking Protocol (K8s 1.24+)

[App.] Health Checks - Parameters

- Configuration parameters
 - `initialDelaySeconds` – wait before first probe (default 0)
 - `periodSeconds` – probe interval (default 10s)
 - `timeoutSeconds` – probe timeout (default 1s)
 - `failureThreshold` – consecutive failures before action (default 3)
 - `successThreshold` – consecutive successes to recover (default 1)
- Common pitfalls
 - `initialDelaySeconds` too short → repeated probe failures / restart loop during startup
 - `timeoutSeconds` too short → flapping probes under load
 - No Readiness probe → 502/503 errors during rolling update

[App.] Health Checks – YAML Manifest

- readinessProbe controls traffic
- livenessProbe controls restart

```
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: healthcheck-demo
5 spec:
6   containers:
7     - name: healthcheck-demo
8       image: 'registry.k8s.io/busybox:1.27.2'
9       args:
10         - /bin/sh
11         - '-c'
12         - touch /tmp/healthy; sleep 30; rm -f /tmp/healthy; sleep 600
13       livenessProbe:
14         exec:
15           command:
16             - cat
17             - /tmp/healthy
18           initialDelaySeconds: 5
19           periodSeconds: 5
20       readinessProbe:
21         exec:
22           command:
23             - cat
24             - /tmp/healthy
25           initialDelaySeconds: 5
26           periodSeconds: 5
```

Resource Management

Including: Requests & Limits, QoS, LimitRange / ResourceQuota

Resource Requests & Limits

- Two resource types: CPU (compressible) and Memory (incompressible)
 - CPU units
 - 1 = 1 vCPU core; 1000m = 1 core; 250m = 0.25 core
 - Memory units
 - Ki / Mi / Gi (binary, base-1024) vs K / M / G (decimal)
 - Use Mi / Gi to avoid ambiguity
- **requests** – minimum guaranteed; used by the Scheduler
 - Pod is only placed on a Node that has enough free requested resources
 - Does NOT mean the Pod is capped at this amount
- **limits** – hard maximum; enforced by Linux cgroups at runtime

Resource Requests & Limits

- Behavior when limits are exceeded
 - CPU: container is THROTTLED – slowed down, NOT killed
 - CPU throttling can cause unexpected latency spikes
 - Memory: container is OOMKilled – kernel kills it immediately
 - `kubectl describe pod` or container last state shows OOMKilled
 - Container restarts if restartPolicy: Always
- Tip: set CPU limit with care; CPU throttling is often misdiagnosed

Quality of Service Classes (QoS)

- Quality of Service class determines eviction priority under memory pressure
- **Guaranteed** – requests == limits for ALL containers in Pod
 - Highest priority, last to be evicted
 - Best for latency-sensitive, stateful workloads
- **Burstable** – at least one container has requests $\neq$ limits
 - Middle priority, evicted after BestEffort and before Guaranteed
- **BestEffort** – no resource requests or limits set
 - Lowest priority, evicted first under pressure
 - Never set in production

LimitRange and ResourceQuota

- **LimitRange** – namespace-level defaults & bounds
 - Sets default requests/limits for Pods that omit them
 - Enforces min/max per container (rejects Pods that violate)
- **ResourceQuota** – namespace-level total cap
 - Limits total CPU / memory / PVCs / object count per namespace
 - Prevents one team from starving the entire cluster
 - `hard: { requests.cpu: "10", limits.memory: "40Gi", pods: "50" }`

[App.] Resource Requests & Limits - YAML Manifest

- Guaranteed QoS
 - requests == limits
- LimitRange sets
 - namespace defaults
 - applied when Pod omits resources

```
1 containers:
2   - name: app
3     resources:
4       requests:
5         cpu: 500m
6         memory: 512Mi
7       limits:
8         cpu: 500m
9         memory: 512Mi
```

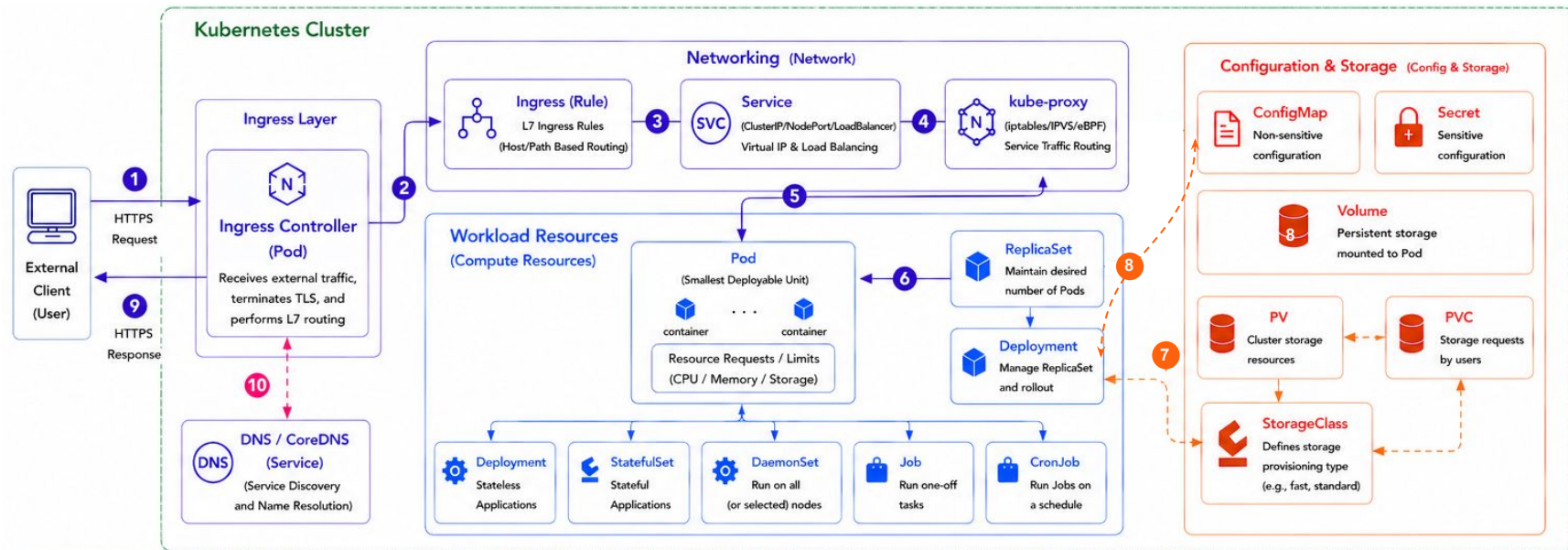
```
1 apiVersion: v1
2 kind: LimitRange
3 metadata:
4   name: default-limits
5   namespace: dev
6 spec:
7   limits:
8     - type: Container
9     default:
10       cpu: 200m
11       memory: 256Mi
12     defaultRequest:
13       cpu: 100m
14       memory: 128Mi
15     max:
16       cpu: '2'
17       memory: 2Gi
```

[App.] Resource Requests & Limits - YAML Manifest

- ResourceQuota
 - hard limits total usage in namespace
 - enforced at create time, not runtime

```
1 apiVersion: v1
2 kind: ResourceQuota
3 metadata:
4   name: team-quota
5   namespace: dev
6 spec:
7   hard:
8     pods: '50'
9     requests.cpu: '10'
10    requests.memory: 20Gi
11    limits.cpu: '20'
12    limits.memory: 40Gi
13    persistentvolumeclaims: '10'
```

Summary



[App.] Request workflow

1. User sends request

- User initiates an HTTP/HTTPS request to the application domain/IP

2. Request enters Ingress Controller

- Traffic reaches the Ingress Controller which terminates TLS and applies Ingress rules

3. Ingress routes to Service

- Ingress Controller matches the rule and forwards the request to the target Service (ClusterIP)

4. Service load balances

- Service selects a healthy backend Pod IP and forwards the traffic via kube-proxy

5. Traffic reaches Pod

- kube-proxy (iptables/IPVS/eBPF) routes the traffic to the selected Pod through the CNI network

[App.] Request workflow

6. Pod processes request

- The containers inside the Pod process the application request within resource limits

7. Storage access (if needed)

- Pod reads/writes data using Volumes backed by PV/PVC (if the application needs persistent storage)

8. Configuration access (if needed)

- Pod reads configuration data from ConfigMap and/or Secret

9. Response back to user

- The response travels back: Pod → Service → Ingress Controller → User

10. DNS resolution (if needed)

- If the user accesses via domain, CoreDNS resolves the domain name to the Ingress IP